

Adopting the Kit

Installing the workflow into a real project, step by step

You have decided to run a project the way `WORKFLOW_GUIDE.md` describes, and the rationale guide, `KIT_RATIONALE.md`, has told you why the kit is shaped the way it is. This guide is the practical part. It walks you through installing the kit into a real project, from the first command to the point where the project has begun growing its own structure as the work proceeds. Everything here happens inside your project, and the kit writes only generic rules and empty structure, never your data.

What you need

You need three things, and none of them is unusual. First, the kit itself: the `planner-kit` directory, which holds the installer, `install.sh`, with a `payload` folder beside it. It ships inside the Claude Code toolkit, at `CCRT/planner-kit/`. Second, a project to install into, which is any directory you want to serve as the project root. Third, a shell: the installer runs on `bash 3.2` and later, the macOS default, and relies on nothing exotic. Before you start, note where the kit lives on your machine, because you will name that path when you run it.

The kit and the toolkit around it install separately, on purpose. Installing the CCRT once into `~/.claude` equips every session with the global capability, the agents, skills, rules, and hooks that apply everywhere. This kit's own installer runs in each project root that adopts the supervisory workflow and writes that one project's operating contract. The two installers are deliberately separate, and they are designed to be used together. To bring an already-installed project up to the current rules later, re-run the installer with `--upgrade-rules`.

One habit matters more than any of this. Run the installer from your project root, not from inside the kit. The installer will refuse to run against itself, for the reason the rationale guide tells, but the point is to stand in the project you are installing into, so that the current directory is the target.

The two install modes, and when each fits

The installer has two modes, and the default is the minimal one. In minimal mode it installs two files at your project root, the rules file (`CLAUDE.md`) and the folder contract (`STRUCTURE_RULES.machine.md`), plus the two advisory hooks described further down and the model-routing set, two executor agents and a model-verification skill under `.claude/`, and it creates no folders; the project's tree grows on demand afterward, as tasks call for it. In full mode, which you request with `--full`, it also builds the entire standard tree up front and seeds every template, which is the classic layout that some people prefer to see all at once.

Choose the minimal default for a fresh, long-lived project whose eventual shape you cannot predict; keeping the tree empty until the work fills it is the whole point of the default. Choose `--full` when you want the complete layout visible from the start, or when you are bringing over a project that already expects the classic structure. The two commands are these:

```
cd /your/project && bash /path/to/planner-kit/install.sh          # MINIMAL: 2
root files + hooks, folders on demand
```

```
cd /your/project && bash /path/to/planner-kit/install.sh --full      # CLASSIC:
pre-scaffold the whole tree + seeds
```

If your project path or the kit path contains spaces, quote it, because an unquoted space will break the command:

```
cd "/your/project" && bash "/path/to/planner kit/install.sh"
```

Preview it first with a dry run

Before you change anything, run the installer with `--dry-run`. It writes nothing and prints exactly what it would do, one action at a time, marking each as a folder it would create, a template it would seed, or a marker it would add. The dry run composes with either mode, so you can preview the full install as readily as the minimal one:

```
cd /your/project && bash /path/to/planner-kit/install.sh --dry-run
cd /your/project && bash /path/to/planner-kit/install.sh --full --dry-run
```

What the preview shows you is trustworthy in a specific, designed way. The dry run and the real run decide what to write from one shared piece of code reading the same inputs, so the list the preview prints is precisely what a real run will write. That shared decision is why the rationale guide can call the preview honest: the preview is the same decision the real run makes, run once without writing anything. Read the summary, confirm it matches what you expect, and then run the same command again without `--dry-run`.

What the two files are

When the install finishes, two files sit at your project root, and it is worth knowing what each one does. The rules file, `CLAUDE.md`, is the front door. It carries the whole operating contract: the supervised loop and the standing disciplines that keep it honest. It installs at the root, rather than inside the `.claude` folder, because external code-review and continuous-integration runners look at a repository's root, and a short pointer stub is left at `.claude/CLAUDE.md` so a reader who looks there is sent to the root file.

The second file, `STRUCTURE_RULES.machine.md`, is the folder contract, and its reader is the coordinating agent rather than you. It lists every folder the project may grow, and for each one it gives the folder's purpose, the trigger that calls it into being, the rule that governs it once it exists, and the mechanical steps to create it. You rarely need to read it yourself. It is the reference the agent consults each time the work reaches for a folder that is not there yet.

How the agent grows the project

This is the part that feels unlike an ordinary scaffold. Because the minimal install creates no folders, the coordinating agent builds each one as the work asks for it, using the folder contract to know when and how. When a task first needs to persist a brief, the agent reads that the exchange folder is called into being by exactly that need, creates it, and carries on. When it first needs a ledger, it copies the ledger template into place at that moment; when it first retires a document, it brings in the retirement tool the same way. Each folder appears the instant its trigger fires and not before, and a folder that never appears records that the project never needed it.

Your first working cycle

With the two files in place, you can begin working exactly as `WORKFLOW_GUIDE.md` describes, and this guide will not repeat that loop. What is worth watching, the first time through, is the tree filling itself in. You hand the coordinating agent a goal. It plans, and to brief its first subagent it persists that brief to a file, which is the moment the exchange folder and its briefs subfolder come into existence. The subagent does its work and writes an output, and if that output is a report, a reports folder appears to hold it. The first time the agent records that something was measured, it copies in the efficacy ledger; the first time it snapshots a plan, the plans folders appear under their status names. After a few cycles the project has precisely the structure its work has required, and nothing beyond it. You did not lay that structure out in advance; the work and the folder contract laid it out between them.

One thing about those launches is worth knowing before the first one, because it reads one setting deep. The subagents carry no model setting of their own, so the coordinating agent chooses each one's model as it launches it, naming the tier it wants, top tier included. Measure your own runtime's precedence before you rely on any of this. On the runtime this kit was built against, four settings decide a subagent's model and the highest one in force wins: an environment variable for the subagent model, then the model name on the launch itself, then the agent file's own model field, then whatever the session runs on. A launch that names nothing therefore asks for the session's model rather than the top of the band. An earlier version of this rule advised the opposite, that you request the top tier by naming nothing, on grounds now known to be mistaken: that naming it could silently reach an excluded model. The excluded model did arrive, but through a substitution on the serving side that naming or omitting does not control, so verify a model claim from what the serving side wrote on the worker's own transcript rather than from the launch settings or from the worker's own account of itself.

The two hooks that watch the work

Both modes also put two small hooks into `.claude/hooks/` and register them in `.claude/settings.json`. If you have no settings file, the installer seeds one; if you do, it deep-merges into it, keeping a dated copy of your original first, and it matches on the command, so a re-run registers nothing twice.

What they do is easy to recognize when it happens. Launch a subagent against a brief that still has an empty slot, and the first hook names that slot as the launch goes out. End a turn in which you collected subagent results without naming one of the six collect outcomes `WORKFLOW_GUIDE.md` defines, `CONTINUE`, `RE-ROUTE`, `FIX-FIRST`, `ABORT`, `GOAL-MET` and `ADAPT`, and the second asks you for it, once. That second nudge costs one extra turn each time it fires, because the interface it hangs on offers no advisory channel. Neither hook ever refuses an action, both let the work through if anything inside them breaks, and `PLANNER_KIT_HOOKS=off` silences both.

What the first hook reads is the brief form, which arrives with the briefs folder: the agent copies `_TEMPLATE.md` in the first time it persists a brief, and `--full` seeds it up front. Working from it is three steps. Copy it to `dev/briefs/<ID>-<slug>.md`, fill every slot, then launch; an empty slot means the brief is not ready, which is the whole bar. The form ends with a sub-planner block you

delete unless that subagent will coordinate subagents of its own. When it will, that block is where you name the subtree it stays inside, the private area `dev/briefs/<ID>/` those subagents' briefs go to, and the rolled-up report you expect back.

Upgrading a project from an earlier version

If you are installing into a project that already carries the kit from an earlier version, the installer helps you upgrade without ever touching your content. Re-running it prints a loud warning that names the version it found and the version it now is, because the two differ. It warns rather than proceeding quietly for a concrete reason: an older rules block will not mention the files this version adds, such as the folder contract, so a freshly seeded folder contract could sit at your root unreferenced by the older rules beside it. The merge itself still does nothing to your file.

To finish the upgrade, do the single manual step the warning names. Delete the whole marked block, from its `<!-- planner-kit:BEGIN ... -->` marker through its `<!-- planner-kit:END -->` marker, out of your `CLAUDE.md`, and run the installer again to write the current block in its place. There is an older situation the installer also recognizes: if it finds the kit's rules living inside `.claude/CLAUDE.md`, where the earliest layout kept them, it prints migration advice and leaves that file exactly where it is, so the choice of whether and when to move it stays yours.

Re-running is always safe

You never have to worry about running the installer more than once. A re-run changes nothing it has already done: it deletes nothing, overwrites nothing, and seeds a template only when that template is absent, so a folder contract you have edited by hand survives a re-install intact. Once the marked rules block is present in your `CLAUDE.md`, running the installer over it again does nothing. This safety is deliberate, and it is what lets you preview, install, and later re-install after an upgrade without needing a rollback between the steps, though the workflow keeps dated backups anyway, for the reasons `WORKFLOW_GUIDE.md` gives.

Where the records live as the work accumulates

As the project runs, a few homes fill up, and knowing where they are lets you read the state of the work directly, in files you can open. The exchange layer, `dev/`, holds the briefs, the instrument scripts, the reports, and the three ledgers: the efficacy ledger that tells you what has been measured rather than merely built, the append-only change log that records everything that has happened, and the code inventory that keeps agents from rebuilding what already exists. The `plans/` folder holds plan snapshots sorted into active, parked, and finished, alongside a plan ledger that tracks their moves. And `Stale_Trash/` holds retired items, each one moved there only after a tombstone was written into it, and none of them ever read as current again.

You do not have to hold any of this in memory. The folder contract, `STRUCTURE_RULES.machine.md`, is the authoritative description of every folder and the rule that governs it, and the rules file, `CLAUDE.md`, carries the disciplines those rules refer to. Between them, the two files the installer put at your root are enough to teach any fresh agent, and you, how the project is meant to work. Adoption is just those three moves: preview the install, put down the

two files and the hooks that back them, and let the project grow its own structure as the work gives it reason to.